A. Vasya And Password

time limit per test 1 second
memory limit per test 256 megabytes
input standard input
output standard output

Rating: 1200 | Tags: greedy, implementation, strings | https://codeforces.com/problemset/problem/1051/A

Vasya came up with a password to register for *EatForces* — a string *s*. The password in *EatForces* should be a string, consisting of lowercase and uppercase Latin letters and digits.

But since *EatForces* takes care of the security of its users, user passwords must contain at least one digit, at least one uppercase Latin letter and at least one lowercase Latin letter. For example, the passwords "`abaCABA12`", "`Z7q`" and "`3R24m`" are valid, and the passwords "`qwerty`", "`qwerty12345`" and "`Password`" are not.

A substring of string *s* is a string $x = s_l s_{l+1} \ldots s_{l+len-1}$ ($1 \le l \le |s|$, $0 \le len \le |s| - l + 1$). *len* is the length of the substring. Note that the empty string is also considered a substring of *s*, it has the length 0.

Vasya's password, however, may come too weak for the security settings of *EatForces*. He likes his password, so he wants to replace some its substring with another string of the same length in order to satisfy the above conditions. This operation should be performed **exactly** once, and **the chosen string should have the minimal possible length**.

Note that the length of *s* should not change after the replacement of the substring, and the string itself should contain only lowercase and uppercase Latin letters and digits.

Input

The first line contains a single integer *T* ($1 \le T \le 100$) — the number of testcases.

Each of the next *T* lines contains the initial password *s* ($3 \le |s| \le 100$), consisting of lowercase and uppercase Latin letters and digits.

Only *T* = 1 is allowed for hacks.

Output

For each testcase print a renewed password, which corresponds to given conditions.

The length of the replaced substring is calculated as following: write down all the changed positions. If there are none, then the length is 0. Otherwise the length is the difference between the first and the last changed position plus one. For example, the length of the changed substring between the passwords "`abcdef`" → "`a7cdEf`" is 4, because the changed positions are 2 and 5, thus (5 - 2) + 1 = 4.

It is guaranteed that such a password always exists.

If there are several suitable passwords — output any of them.

Example

Input
2 abcDCE htQw27

Output
abcD4E htQw27

Note

In the first example Vasya's password lacks a digit, he replaces substring "C" with "4" and gets password "abcD4E". That means, he changed the substring of length 1.

In the second example Vasya's password is ok from the beginning, and nothing has to be changed. That is the same as replacing the empty substring with another empty substring (length 0).